A Self-Interpreter for SKI Written in SKI: Extensional Semantics for New Symbols

N. E. Davis

September 15, 2026

Abstract

We build a symbolic step-evaluator for SKI combinatory logic as a closed term of the basis {`S, K, I`}: 618 atoms, all recursion through `Y`, no host support past the one-time encoding of the input. Run on an encoding of itself running on a base term, it reaches weak head normal form in 504,930 contractions and agrees with the host's own reduction. The artifact is small enough to be read whole, and three things about it are worth reporting. First, a structural census over the {`B, C, K, W`} basis shows the representation layer is affine (weakening only) while the interpreter core needs contraction at exactly seven variable occurrences; representation needs weakening, iteration needs contraction, and contraction alone already costs normalization. Second, extending the object language with a fifth constructor forces the interpreter's result type to grow, and whether the new outcome is exposed as a value or absorbed into host divergence is a single-arm edit at one named site, measured at four atoms versus seven in this file, with the two interpreters provably agreeing on every input that never reaches the site. We read this as a minimal executable instance of metacircular evaluation in the sense of Nock's `+mock`, and we state precisely where the analogy breaks: Nock's virtualization is free because formulas are nouns, while SKI must pay one external quoting step, which no SKI term can perform (Jay & Given-Wilson 2011, Thm. 3.2). Third, we build the analogy out: a nonlocal scry site in the style of opcode 12, with the resolver passed as an argument; a blocking version with a namespace that grows across re-runs; and a namespace over arbitrary compound paths keyed by a 240-atom structural-equality combinator, which corrects an earlier claim that such a namespace was out of reach.

Metacircular evaluation permits the separation of the evaluation mechanism from the object language, allowing for controlled experimentation with different evaluation strategies and the observation of intermediate states without affecting the host environment. An interpreter can be employed to handle states such as partial evaluation without interfering with the host environment, particularly in cases in which the host environment itself may not be capable of representations such as partially evaluated terms, intermediate representations of the object language, crashes, and other exceptional states.

We construct a symbolic step-evaluator for the SKI combinator calculus as a closed term of the basis {`S, K, I`}. This evaluator performs symbolic evaluation by repeatedly applying the reduction rules of the SKI calculus until a normal form is reached. Because the interpreter is capable of symbolic representation beyond simply the basis {`S, K, I`}, we can add in functionality such as handling additional combinators, supporting custom reduction strategies, and managing exceptional states like crashes, undefined terms, or as-yet-unknown referentially transparent lookups.

1 Metacircular Evaluation

While many languages nominally support it, metacircular evaluation as a first-class strategy has only occasionally been applied outside of the LISP family of languages, where it has been used to implement features such as macro systems, partial evaluation, and sandboxed execution environments. Essentially, each nested interpreter acts as a virtual machine for the next level of interpretation. LISP's homoiconic nature is particularly apt for such virtualization, since the interpreter can directly treat its interpretand as data, enabling LISP's powerful reflective and metaprogramming capabilities.

The lineage runs from McCarthy [1960], whose universal function `eval` was written in the language it evaluates, and from the Lisp 1.5 manual [McCarthy et al., 1962], which specifies the language by that function. Reynolds [1972] gave the form its critical treatment: a definitional interpreter written in the style of its own object language is metacircular, and it inherits the defining language's evaluation order unless that order is made explicit. The present work meets that observation directly in §5, where the object-level laziness of `K` is inherited rather than implemented. Guy L. Steele and Sussman [1978] and Abelson et al. [1996] made the metacircular evaluator a standard instrument for experimenting with semantics, which is the use it is put to here.

The Nock combinator calculus explicitly adopted self-application as a fundamental mechanism for metacircular evaluation from its earliest inception. This mechanism was originally intended as a "hyper-Turing" operator [Davis, 2025], in which referentially transparent queries could answer with a value, a permanent unavailability, or a deferral.

The tower as such enters with Smith [1984], whose 3-Lisp runs each level of a reflective tower on an interpreter at the level above; Wand and Friedman [1988] gave the tower a non-reflective description, and Asai et al. [1996] the language Black, in whose style Amin and Rompf's Purple is cast.

Amin and Rompf [2018] treat the tower itself as the object of study. Their multi-level λ-calculus provides staging constructs and stage polymorphism, so that a single evaluator acts as an interpreter or as a compiler depending on runtime parameters, and their metacircular evaluator Pink collapses arbitrarily many levels of self-interpretation into one compiled program; correctness of self-interpretation and multi-level Jones optimality are stated without formal proof but backed by experiment (their Propositions 4.2 and 4.6). Semantic modifications introduced at any level of the tower, such as instrumentation, CPS conversion, or reflection in the style of Black, survive the collapse. Their objective is performance, the removal of interpretive overhead, and their semantics is value-producing throughout: there is no fuel, no failure, and no outcome that has to be represented rather than computed. The present work is close to complementary. We keep the levels and pay for them (§7), and we ask what an interpreter does at the site where its object language gains a construct whose meaning the interpreter must author (§6), a question that does not arise in a setting without a third outcome. What the two share is the shape of the mechanism: an interpreter that is an ordinary program in the calculus it interprets, with one place at which its behaviour is substituted, which is the shape §8 builds on.

In the λ-calculus proper, Barendregt [1991] states the requirement on a self-interpreter, $\mathbf{E}\ulcorner M\urcorner = M$, and notes that the coding $\ulcorner\cdot\urcorner$ is a metalevel operation rather than a term; that boundary is the one §3 locates for SKI with Jay and Given-Wilson's theorem. Berarducci and Böhm [1993] show that the choice of encoding is load-bearing, obtaining a self-interpreter that itself has a normal form, and Brown and Palsberg [2016] carry self-interpretation into System F_ω, where the barrier is typing rather than quotation. The specialization of an interpreter to a fixed

program is Futamura's first projection [Futamura, 1971]; Jones et al. [1993] is the standard account, and Jones [1996] names the optimality condition a specializer must meet to remove interpretive overhead entirely, a condition Amin and Rompf claim and this work does not.

The question which motivated the current line of research was how amenable the SKI combinator calculus is to metacircular evaluation, and what design patterns and implementation techniques could facilitate such evaluation. We do not anticipate such a line of inquiry to be competitively efficient versus other combinator systems or higher-level representations, but rather as a tool for exploring the metacircular evaluation process itself.

2 The artifact

`tower_harness.py` defines the interpreter by named Scott-encoded definitions and expands them to closed {`S, K, I`} terms. Sizes are tree-atom counts (`aviary_kernel.terms.size`, which counts atoms and not application nodes):

component	tree-atoms
`whnfF`, the emulator	618
`UQ`, the unquoter	43

The script's own banner prints 348 and 62. Those are counts of *DAG nodes* (atoms plus application nodes, deduplicated by identity, from the local `size()` at lines 24 to 36), not tree-atoms, which is why the second number is larger than 43 rather than smaller. Both are correct measurements of different things; the banner does not yet say which.

Targets, all `VERDICT: OK` against the host reducer:

- **T0** `whnfF 3 <I K>`: 340 contractions.
- **T1** the emulator over the unquoter: 91,556 contractions.
- **T2** the tower, the emulator over an encoding of itself over `I K`: 504,930 contractions. The script's default object fuel of 120 is below the 341 the inner run requires, and the script says so; any `--kk` of at least 341 gives this count (run at 341 and at 500).
- **T3** the observability pair: §6.

`ski-in-ski.ipynb` executes end to end with zero errors and prints the 618, 43, and 504,930 figures in its stored outputs (the notebook file now stores its executed outputs). `ski-in-ski-aviary.ipynb` builds the same interpreter natively in the `aviary` kernel and executes with zero errors, but measures no atom counts and runs no tower; it is a construction walkthrough, not a second reproduction of the figures.

Three follow-on files extend the object language with a fifth leaf, `Scry`, and are the subject of §8. Each loads the previous file's environment and modifies nothing in it, the pattern `pipeline.py` established:

component	file	tree-atoms
`wfQ`, oracle-threaded emulator	`scry_harness.py`	950
`wfN`, blocking emulator	`scry_namespace.py`	1,066

component	file	tree-atoms
`EQ5`, structural equality on encoded paths	`scry_paths.py`	240

All three scripts are self-verifying: each printed claim is checked against a fresh-marker probe of the reduced value, and each exits 1 if any check fails (confirmed by injecting a wrong expectation into a copy).

3 Encoding is external, decoding is internal

`S K K` and `I` agree on every argument, yet as terms of weak combinatory logic they are distinct normal forms, 3 atoms against 1. A `quote` function from live terms to Scott-encoded data must tell them apart, and no SKI term can. Jay & Given-Wilson, *A combinatory account of internal structure* (JSL 76(3), 2011), Theorem 3.2, §3: factorisation of SK-combinators is a symbolic computation that is not representable. The theorem is stated for {`S, K`}; its proof goes through unchanged with `I` primitive. Quotation follows from it in two lines. If `Q M` $\twoheadrightarrow \langle$`M`$\rangle$ were definable, then under the translation to λ-calculus $[\![$`Q`$]\!]$ `(λx.x)` is one fixed term, yet `Q (S K X)` yields $\langle$`S K X`$\rangle$, from which `X` is extractable by Scott dispatch, so every `X` would share a reduct, contradicting that `S` and `K` are distinct normal forms. (Equivalently, `Q` together with this artifact's own `UQ` would build a factorisation combinator, contradicting Theorem 3.2 directly.)

This is why `enc()` and `encQ()` are Python. Everything on the other side of that boundary, `UQ`, `whnfF`, their self-application, and everything in §8 including path equality, is ordinary {`S, K, I`}. Quoting is a one-time cost paid to enter the representation, and every level above it is closed. `ski-in-ski-aviary.ipynb` makes this its explicit stopping point ("the wall", its §8).

4 Where the structural cost lives

`census.py` and `bckw_census.py` rebuild each component over {`B, C, K, W`} and record which structural rule each needs, weakening (`K`) or contraction (`W`). The measurement that is independent of the abstraction algorithm is the *occurrence* census (`census.py` §1): a component needs weakening iff some binder occurs zero times and contraction iff some binder occurs more than once. The atom census is convention-relative and the scripts say so: with `I` primitive the interpreter core shows 7 `W` atoms, with `I = W K` it shows 10, and under the latter convention even the encoding layer shows 4.

The representation layer is affine. All ten Scott constructors (`enc<S>`, `enc<K>`, `enc<I>`, `enc<app>`, `nil`, `cons`, `nothing`, `just`, `zero`, `suc`) have no binder occurring twice: a constructor selects one continuation and discards the rest, which is weakening by construction. The representation lives in BCK.

The interpreter core needs contraction at five of eight components, seven surplus occurrences in total: `UQ`'s `f` (twice, in `(f t)(f u)`), the spine generator's `acc` (four times), the rebuild generator's `h` (twice), the `S`-step arm's `z` (twice, in `x z (y z)`, which is the `S` rule itself reappearing one level up), and the fuel loop's `m` (twice). Surplus occurrences sum to `1 + 3 + 1 + 1 + 1 = 7`, and any BCKW abstraction must perform at least that many contractions. The differential check (18/18 SKI and BCKW abstractions extensionally agree; the purity guard

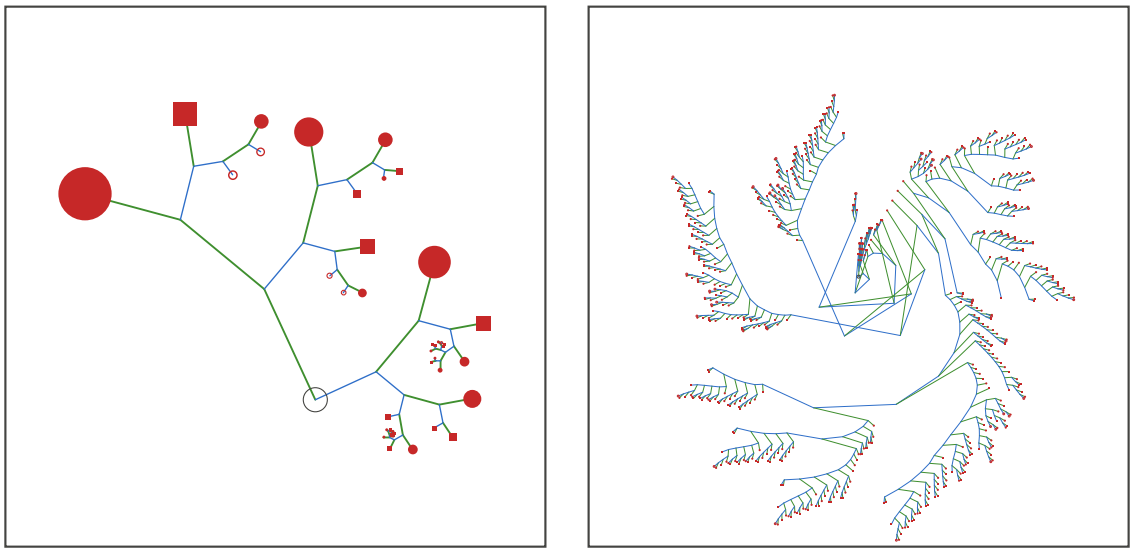


Figure 1: The emulator `whnfF` as a term. Left, the typed-leaf spiral: function edges green, argument edges blue; `S` a filled circle, `K` a square, `I` a ring; the construction rule prunes subtrees below a length cutoff, leaving 48 of the 618 atoms visible. Right, every atom: angle is in-order leaf position, radius is depth, 48 levels. Each fern-like arm is one deeply nested argument, which is what bracket abstraction produces.

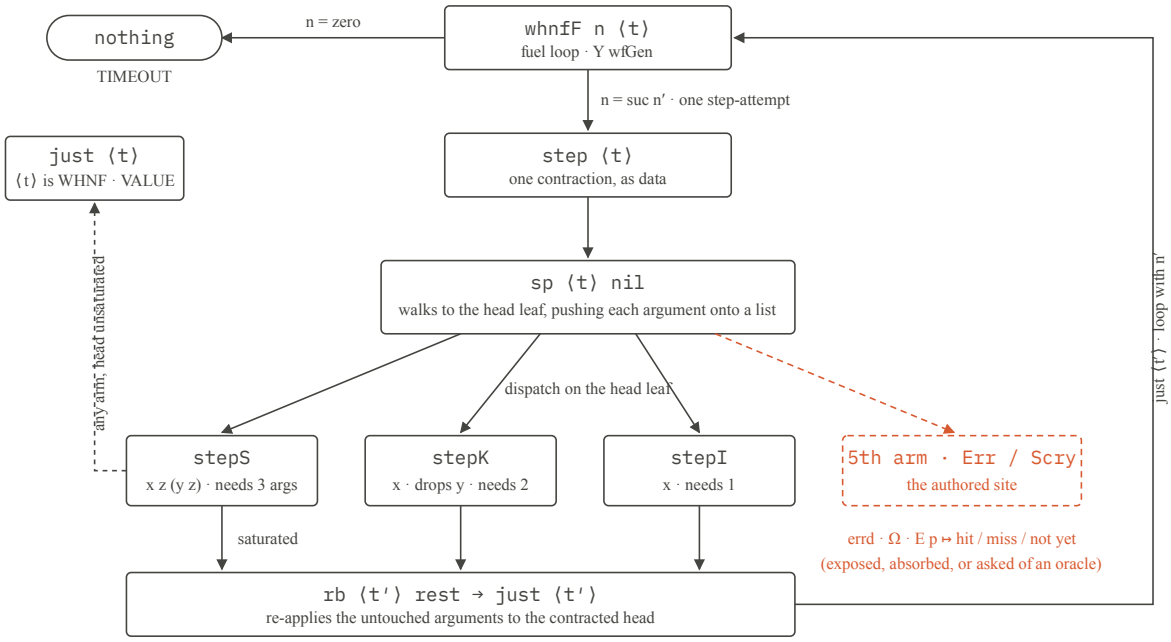


Figure 2: How `whnfF` evaluates. The fuel loop peels one `suc` per step-attempt; `step` walks the spine to the head leaf, collecting the arguments; dispatch selects the arm for that leaf; a saturated arm contracts and `rb` re-applies the untouched arguments, returning `just` ⟨`t'`⟩ to the loop; an unsaturated head returns `just` ⟨`t`⟩, the value; exhausted fuel returns `nothing`. The dashed fifth arm is the site of §6.

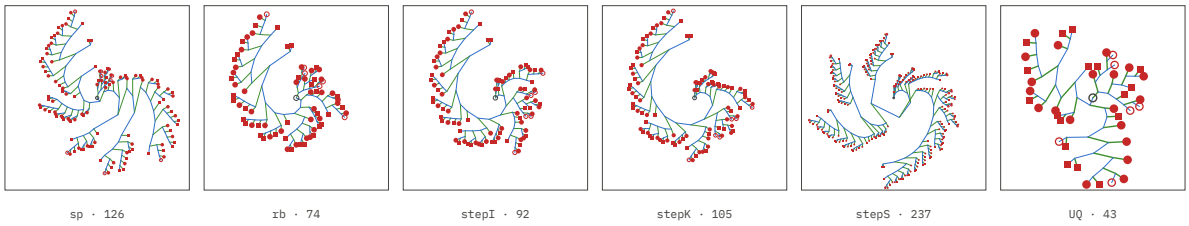


Figure 3: The interpreter's components as terms, every atom drawn: the spine walker `sp` (126 atoms), the rebuilder `rb` (74), the three step arms `stepI` (92), `stepK` (105), `stepS` (237), and the unquoter `UQ` (43). Each panel is scaled to its frame, so sizes are not comparable across panels. The `S` arm is the largest because it must build `x z (y z)` as data.

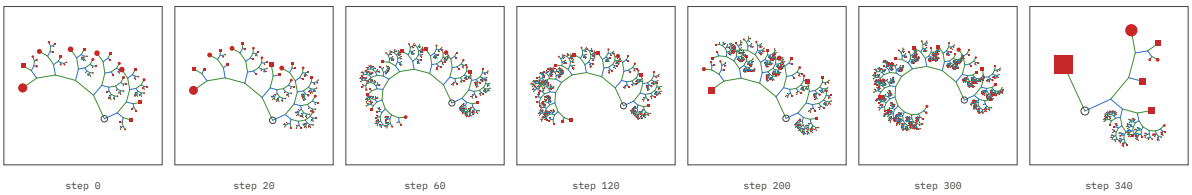


Figure 4: The emulator running T0, `whnfF 3` ⟨`I K`⟩, after 0, 20, 60, 120, 200, 300 and 340 host contractions, in the spiral view. The spine curls as `Y` unfolds and the encoded term is walked; the tree reaches about 111,000 atoms at step 300 and collapses to the answer, `just` ⟨`K`⟩, at step 340.

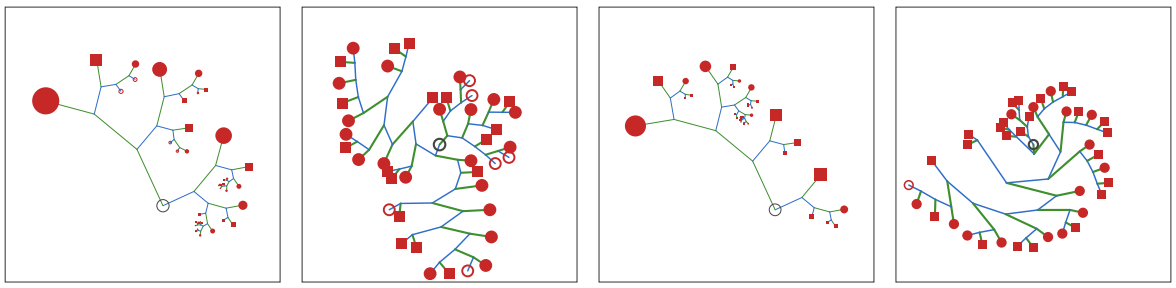


Figure 5: The decoder and a datum. `UQ`, 43 atoms, and the Scott encoding ⟨`I K`⟩ of the two-atom term `I K`, 40 atoms, each in spiral and every-atom views. A datum and a program look alike because in this calculus they are the same kind of thing; only the encoding step that produced ⟨`I K`⟩ lies outside it.

accepts pure-basis terms and rejects mixed ones) confirms the abstractions are correct; it does not bear on the census, and is not cited for it.

Why each rule is needed. Removing weakening removes dispatch: a Scott constructor *is* an n-ary discard. Removing contraction removes iteration: `Y` duplicates its argument, and the affine fragment BCK is strongly normalizing, so it has no fixpoint combinator and no fuel loop. (The fuel itself is a Scott numeral, which is affine; iteration here comes from `Y`, not from the numeral.) Self-interpretation therefore needs both rules for two different reasons. The asymmetry is worth stating exactly: contraction alone already costs normalization. `(S I I)(S I I)` contains no `K`, and in BCKW `W W W` reduces to itself with no weakening anywhere; the relevant fragment BCIW is not normalizing while BCK is. So the rule that buys iteration is the rule that costs normalization, and weakening is bought for representation, not for divergence.

5 Fuel and the third outcome

An unfueled interpreter's outcome space is {`value, divergence`}, and divergence cannot be observed from inside a finite run. Fuel converts it to {`value, timeout`}, both observable. The precise limitation is local: at any fixed fuel `k`, one observation does not separate a divergent term from one that needs more than `k` steps. Ω reports `TIMEOUT` at every `k`; $\mathtt{I}^{31}\mathtt{K}$ reports `TIMEOUT` for $\mathtt{k} \leq 31$ and a value from `k = 32` on (31 contractions, and fuel `k` permits `k` step-attempts of which the last must be the no-redex check). Convergence is semi-decidable by raising `k`; divergence is not. `K I Ω` still discards Ω unevaluated under the emulator, exactly as under host reduction, because the object-level `K` rule fires before the interpreter recurses into the argument: object-level laziness is inherited, not re-implemented.

Amin & Rompf's framework (*Collapsing Towers of Interpreters*, POPL 2018) supports a rich family of semantic modifications across tower levels: instrumentation and CPS transformation (their §4.2), `EM`, `call/cc` and `shift/reset` at user level, reflection à la Black, and meta-level undo (§5 to §7). Their correctness and optimality properties (Props. 4.2 to 4.7, of which 4.5 is a definition) are, in their words, "stated without formal proofs but backed by experimental evidence" (§4.1, p. 11). The word "fuel" does not occur in the paper, and no outcome-type that must grow when the object language gains a construct is discussed. Their collapse is a collapse of value-producing computation; the third outcome is outside the space their equivalences range over.

6 Authored failure: one site

Extend the object language with a fifth constructor `Err`, with no object-level contraction rule. Its Scott encoding is unremarkable, five continuations instead of four. What is interesting is what the interpreter does when the spine walker reaches `Err` in head position, and that is a choice made at one site:

```
D("errArmAbs", ["acc"], a("errd"))      # observable: crash becomes a value
D("errArmOmg", ["acc"], OMEGA)           # hidden: crash becomes host divergence
```

(`tower_harness.py` lines 217 to 218, verbatim.) Expanded, the arms are `K (K (K I))` at 4 tree-atoms and `K (S I I (S I I))` at 7; the two full emulators `wf5Abs` and `wf5Omg` measure 768 and 771 and differ at exactly this one definition site. The measurement is of this file, not

a minimum: the `K` wrapper on Ω is unnecessary, since Ω `acc` diverges as Ω does, and a 6-atom arm reproduces the table below cell for cell (`review/omg6.py`). Even the 4 depends on the abstraction algorithm's η and `I` shortcuts. The honest statement is a single-digit-atom edit at one named site.

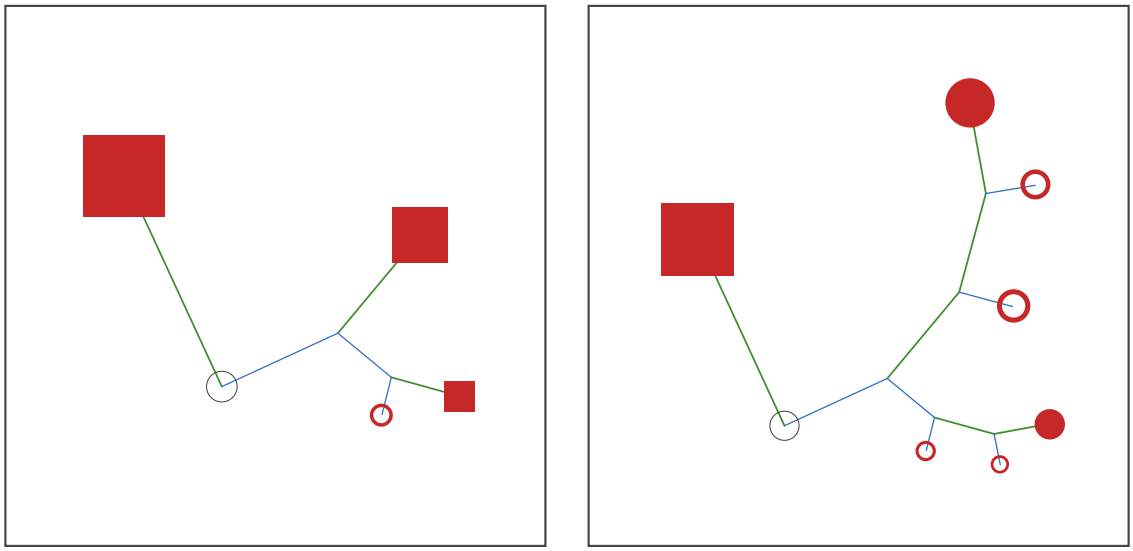


Figure 6: The two arms of the authored site. `errArmAbs = K (K (K I))`, 4 atoms, exposes the crash as a value; `errArmOmg = K (S I I (S I I))`, 7 atoms, absorbs it into divergence. The two emulators differ by exactly these atoms.

Ω `= (S I I)(S I I)` is one of the smallest divergent terms in this basis, not the unique one. Exhaustive enumeration of every {`S, K, I`} application tree of at most 6 atoms (`review/brute2.py`; the counts match $3^n \cdot$ `Catalan(n−1)`, so the enumeration is complete) finds that all 3,873 terms of at most 5 atoms reach a normal form, and exactly five terms of 6 atoms do not, under both "no normal form" and "no weak head normal form": `S I I (S I I)`, `S (S I)(S I) I`, `S S S (S I) I`, `S S I (S I) I`, and `S (S S) S I I`. (Tromp notes that in {`S, K`} alone the smallest unsolvable term is `S S K (S (S S K))`, 7 atoms; consistent with the enumeration, since all five 6-atom divergent terms contain `I`.)

Re-run and verified:

object term	exposing (`errArmAbs`)	hiding (`errArmOmg`)
`Err`	ERR	host diverges
`Err K`	ERR	host diverges
`K I Err`	VAL	VAL
`I K`	VAL	VAL
Ω	TIME	TIME

"Host diverges" is the host reducer's 400,000-step cap being hit; it is not a detected non-termination, though it is certain here because the arm *is* Ω.

Lemma (faithfulness off the site). For every object term that never brings `Err` into head position during its reduction, `wf5Abs` and `wf5Omg` produce the same result. The two definitions differ only in `errArm`, which is referenced only from the `Err` arm of the step function, which the spine walker selects only when the head leaf is `Err`. On any other input the two are the same program. The table's `K I Err` row is the instance where `Err` is present but discarded by object-level laziness before it is ever in head position.

Adding a constructor does not add a meaning. The meaning is authored downstream by whoever writes the interpreter, and here the whole decision is legible as a one-arm diff with a proof that nothing else changed.

7 Virtualized evaluation, read against `+mock`

Nock's `+mock` is not interesting because Nock can interpret Nock; that follows once a formula computes `nock`, since a formula already is a noun. Its interest is that a formula can be run through a *substituted* interpreter, one that resolves a scry differently or catches a crash and returns it as a `unit`, while agreement with the reference interpreter holds wherever the substitution is not exercised, and the substitution is confined to a named site.

Read that way, §6 is the SKI analogue of `+mock`, not §2's tower. The tower demonstrates self-interpretation and fidelity; a general version of that demonstration exists for value-producing towers (Amin & Rompf, Props. 4.2 and 4.6). What `errArmAbs` versus `errArmOmg` demonstrates is the thing `+mock` is for: the same program, the same interpreter apart from one arm, different observable outcomes because that arm decides what one outcome category is allowed to be. The lemma in §6 is the faithfulness requirement, virtualized-with-no-substitution equals real, and `pipeline.py`'s Stage 6 check and T0 to T2 are its instances.

Where the analogy breaks. `+mock` never needs a quoting step, because a Nock formula already is the noun handed to the alternate interpreter. SKI is not homoiconic: `S`, `K`, `I` are opaque, and §3 is precisely the statement that no SKI term can turn an arbitrary SKI term into data about itself. So SKI-in-SKI is not symmetric the way Nock-on-Nock is. It is one unavoidable non-SKI quoting step, and only above that boundary is virtualization SKI-native and closed under further quoting. Identifying which half of "virtualized evaluation" transfers to a non-homoiconic calculus is worth more than the analogy alone.

Depth is bounded for the ordinary reason. Layer one costs 340 contractions to emulate a 1-step reduction; layer two costs 504,930, about 1,485 times layer one. That is the familiar roughly-$10^3\times$ per-layer overhead of naive self-interpretation, not a measurement this artifact is claiming (`GATE1-RELATED-WORK.md`, A3). A third level was not attempted. Real Nock-on-Nock virtualization has the same character, which is why jets exist.

8 Substitutability as an argument: the scry site

8.1 Synchronous: `scry_harness.py`

§6's substitution is a compile-time choice between two source variants. Real scry substitution takes the resolver as an argument. `scry_harness.py` builds that: a separate 5-symbol encoding {`S, K, I, App, Scry`} (parallel to {`S, K, I, App, Err`}, not stacked on it; `Scry` has arity 1

like I), and a fueled loop wfQ E n <t> whose extra argument E is an ordinary Scott-encoded {S, K, I} partial function from paths to Maybe values. When the spine walker reaches Scry p it applies E to p. A hit splices the answer in and reduction continues. A miss is kept as a third, distinguishable step outcome, reusing T3's stepped/done/errd, rather than folded into the pre-existing Maybe, which would conflate "the oracle had no answer" with "genuinely stuck". At the outer result type the three points are {value-or-stuck, miss, timeout}: done maps to rVal, so stuck and value share a tag there, and the stuck/miss distinction is a step-level fact.

The path reaches the oracle as Scott-encoded data, exactly as written and unreduced. Verified against three oracles (answer every path by echoing it; answer none; dispatch on the path's head leaf and answer only K):

test	result
I K, no Scry, either oracle	matches base whnfF exactly
I (Scry K), echo oracle	hit; decodes to the answer
I (Scry K), never oracle	miss, distinct from stuck
Scry alone, unsaturated	stuck (done), neither hit nor miss
K K (Scry K), never oracle	reduces to K; the site is never consulted
Scry K / Scry (S K K), head oracle	hit / miss, path-dependently
Scry (I K), head oracle	miss: I K is not the path K (a probe in this session; not among the script's own checks, which scry_paths.py's syntactic-versus-semantic rows cover)
Ω, fuel exhausted first	timeout, not a miss

The laziness row is the same mechanism as K I Ω in §5, now for a nonlocal effect: a scry site inside a branch the program never forces is never consulted. Substitutability does not require eagerness.

8.2 Blocking: scry_namespace.py

A resolver that always answers is still not +mock's scry: a real lookup can fail to answer *yet*. scry_namespace.py builds the cheapest faithful version, option (a) of SCRY-BLOCKING-DESIGN.md, written before any of it was implemented. The resolver's type grows from 2-way Maybe to 3-way (oJust/oNothing/oNotYet) and the interpreter's outcome types grow to match (pendingN at the step level, rBlockN at the result level). This is the phenomenon of §6 again, deliberately re-triggered by our own extension rather than discovered.

Resuming a block captures no continuation. Because reduction is pure and deterministic, the driver re-runs the whole term from the top under an oracle that now also answers the missing fact. Two things make that sound, and both need saying. First, the re-run's steps *strictly before* the block are identical to the first run's only if the new oracle agrees with the old one on every path the old one answered; that holds because the namespace is append-only, and an oracle that revised an answer would break it. Second, re-running is not fuel-neutral. Each round replays the previous one and then takes at least one more step, so with a fixed fuel budget a block can turn into a timeout on the next round:

```
Scry (S K), fact (S K)=I, fuel=1 -> [(1, 'BLOCK on S K'), (2, 'TIME')]
```

```
Scry (S K), fact (S K)=I, fuel=2 -> [(1, 'BLOCK on S K'), (2, 'VAL')]
```

Recomputation is charged for. The two heavier resume strategies in the design note, resume from the top-level term and full CPS (the move Amin & Rompf's CPS evaluator makes, their §4.2.2), remain unbuilt.

What is SKI and what is host, stated plainly: `wfN` and each round's oracle term are closed {`S, K, I`}; the fact table standing in for an external source of truth, the resume loop, and the "stuck" detection are Python.

The demonstration that matters is that the namespace accumulates. With an empty namespace and a fact table {`S` ↦ `Scry K, K` ↦ `I`}, `Scry S` blocks on `S` (round 1), learns it, re-runs, and the newly substituted `Scry K` blocks on `K` (round 2), a fact forced by the result of resolving the first, before round 3 reaches the value `I`. A path with no entry blocks into a distinguishable stuck state; fuel exhaustion stays distinguishable from a block at every round.

8.3 Compound paths: `scry_paths.py`

`scry_namespace.py` dispatched on a path's head leaf only, so its namespace could not tell `S K` from `S K K`. An earlier draft of this paper, and that file's docstring, said this was forced: that comparing arbitrary encoded paths was "the structural-equality problem Jay & Given-Wilson show plain SK cannot solve." That was wrong, and the review caught it. Theorem 3.2 is about factorising *live* combinators. The path the oracle receives is Scott-encoded *data*, a tree with five leaf tags, and equality on such data is an ordinary structural recursion. Jay & Given-Wilson say so themselves (preprint pp. 7 to 8): once a term is encoded, factorisation "is a routine list operation." What SK cannot do is produce the encoding from a live term, which is §3, and it is the only thing SK cannot do here.

`scry_paths.py` builds `EQ5`, structural equality on the five-constructor encoding, at 240 tree-atoms in {`S, K, I`}, and an oracle that compares the incoming path against each known path in turn:

```
p |-> EQ5 p <k1> (oJust <a1>) (EQ5 p <k2> (oJust <a2>) (... oNotYet))
```

It runs inside `wfN` unchanged. Forty-one checks pass, among them:

test	result
`EQ5 <S K K> <I>`	false (the pair §3 turns on)
`Scry (S K)`, fact `(S K)` ↦ `I`	block, then `I`
`Scry (S K)`, only fact `(K S)` ↦ `I`	block, stuck; no false hit
`Scry (S K K)` vs `Scry (S K)`, facts for both	`K` and `I` respectively: a path and its own prefix are distinguished
`Scry (K I)`, facts `(K I)` ↦ `Scry (S K)`, `(S K)` ↦ `I`	three rounds, chained through a compound answer
`Scry (I K)`, only fact `K` ↦ `S`	stuck: paths are compared as written, not up to conversion
`Scry (I K)`, fact `(I K)` ↦ `S`	`S`

The last two rows are the design point. Paths are intensional, syntactic objects, compared by structural equality of their encodings, exactly as a Nock scry path is a noun compared by noun

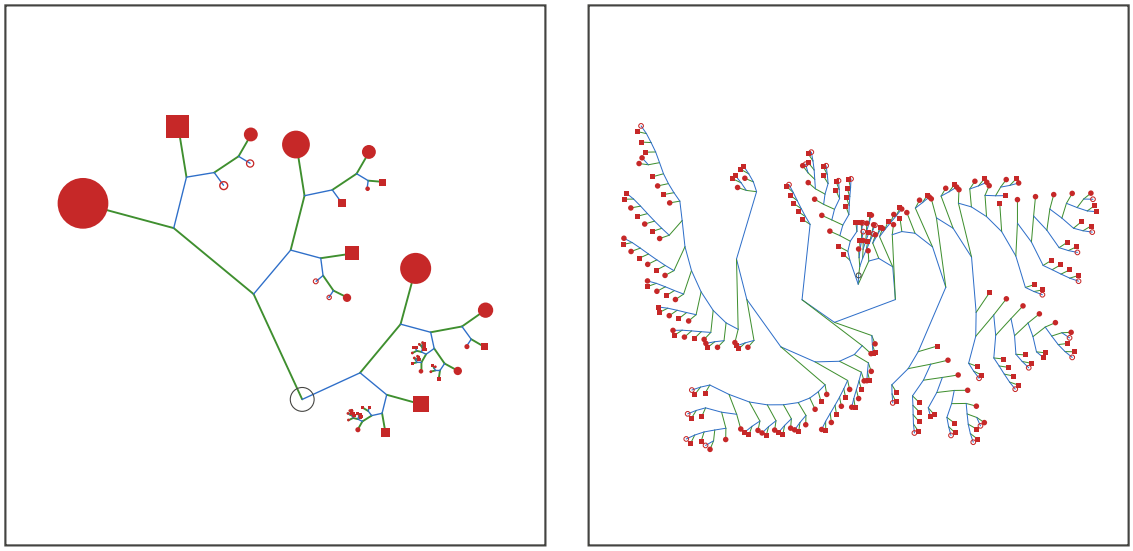


Figure 7: `EQ5`, structural equality on encoded paths, 240 atoms: 92 `S`, 118 `K`, 30 `I`. Spiral view, 68 atoms visible, beside the every-atom disc.

equality. Matching paths up to conversion would be the word problem for SKI, undecidable; normalizing the path under fuel before lookup would be a fueled approximation to it and would reintroduce the third outcome inside path resolution. Nothing built here does either. A three-fact compound-path oracle term measures 1,022 tree-atoms; the driver's decoder from an encoded payload back to an object term, needed to consult the Python fact table, is a behavioral fresh-marker probe, and its round trip is checked.

9 Prior art, and what is claimed

Self-interpretation for a Turing-complete system is old. Kleene 1936 gives the Normal Form Theorem, from which a universal function follows; the explicit universal machine is Turing 1936; "partial recursive" is Kleene 1938. Mogensen 1992 builds a compact self-interpreter and self-reducer in untyped λ-calculus over his own term representation. Jay & Palsberg 2011 build a typed self-interpreter in a factorisation calculus in the style of Jay & Given-Wilson, a combinatory calculus though not SK, and their distinction between self-*recognisers* (recover the term from its representation) and self-*enactors* (execute it) is the one this paper needs below.

The closest prior construction for SK is Tromp, *Binary Lambda Calculus and Combinatory Logic* (Dagstuhl Seminar Proceedings 06051, 2006; the 2023 revision at `tromp.github.io` has different sizes and should not be cited for these). Theorem 3 gives a λ-term `F` of size 124 with `F C (`$\tilde{M}$ `: N) = C M N` for the bitstring encoding $\tilde{M}$ of a combinator `M`; its verification is "left as an exercise to the reader." Theorem 4 states, in full, "There is a self-interpreter F for Combinatory Logic of size 263," obtained from Theorem 3 by improved bracket abstraction and a different

fixpoint combinator (`Y''`); it has no proof text beyond size bookkeeping, and the inductive proof in the paper is for the λ-calculus interpreter of Theorem 2. With `C = I`, `F I (`$\tilde{M}$ `: N) = M N` is a self-enactor, and Tromp's §4.1 uses it that way for universality. Architecturally `F` is a decoder: it reconstructs live syntax from bits and hands it to a continuation, and reduction then happens under the ambient calculus's own rules. It carries no object-level `S` or `K` rule in its body, which means it cannot implement a step relation that differs from the host's, and that is exactly what §6's hiding arm does. The paper also contains (pp. 16 to 18) `nfreduce`, "a complete symbolic lambda calculus normal form reducer" over Scott-encoded de Bruijn terms with explicit substitution, used as a 2,288-bit size constant: a symbolic evaluator over encoded data, for λ-calculus, unfueled, not self-applied, and not run.

So the claim is narrow. A symbolic step-evaluator for SKI written in λ-calculus is a textbook exercise, and bracket abstraction carries it into {`S, K, I`} mechanically. What this artifact adds is the concrete term: 618 atoms, fueled, with its step rules as data manipulation rather than re-spliced host syntax, applied to an encoding of itself, and executed to completion with the count reported. On top of that term, §4's census, §6's one-site authorship with its faithfulness lemma, and §8's substitutable, blocking, compound-path scry site are, to the extent of a full read of the closest papers, not found elsewhere. Amin & Rompf's outcome space has no failure case to author (§5); Jay & Given-Wilson's result is about what SK cannot do to live terms, not about what an interpreter that can be written chooses to expose.

10 Adjacent, distinct work

Two lines of work in this same environment operate over a different basis and question, and are context rather than sources:

- **Infinitary normal forms in the pure S-calculus** (companion draft, `Documents/Davis2026b`) studies `CL(S)`, the calculus generated by `S x y z` $\rightarrow$ `x z (y z)` alone, no `K`, no `I`, and writes out in detail (crediting the observation to an aside in Waldmann's thesis, §7.6) that it is infinitarily strongly normalizing: every term has a unique, possibly infinite normal form, a Böhm-tree analogue with no $\perp$ needed. Its slogan, that `K` "purchases computational universality at the cost of infinitary semantic coherence," is the same style of result as §4 here, attributing a capability and its cost to one specific structural rule. Note that the Ω of §6 contains `I` and is not a `CL(S)` term; that paper's divergent example is the pure-`S` term `S S S (S S) S S`, and it is explicit that once `K` is present the Böhm-tree theory needs quotienting and $\perp$. The two papers treat divergence with different tools, denotational there and operational (fuel) here, on different fragments.
- `ssearch` (`Documents/ssearch`) is a computational classification of divergent pure-`S` orbits by the structure of their infinite spine dynamics, in service of the Wolfram `S`-combinator universality question. Same author, adjacent, not a dependency.

11 Open threads

- **Self-quotation above level 0.** Jay & Given-Wilson 2011 makes no claim about it (its §8, "Structure Completeness," is about `S, K, F, eqatom` and pattern matching; the word "quotation" does not occur). If a stronger claim is wanted it needs Jay, *Programs as*

Data Structures in λSF-Calculus (ENTCS 325, 2016) or Jay, *Self-Quotation in a Typed, Intensional Lambda-Calculus* (MFPS XXXIII, 2017; ENTCS 336, 2018), neither read.

- Kavvos, *On the Semantics of Intensionality and Intensional Recursion* (arXiv 1712.09302), and the differential-testing literature remain unread; `GATE1-RELATED-WORK.md` §5 says why each matters.
- `tower_harness.py`'s banner should label its counts as DAG nodes (atoms plus applications), distinct from the tree-atom counts used here.
- The 6-atom hiding arm is verified but not adopted; adopting it would edit the base file, which every follow-on loads.
- `SCRY-BLOCKING-DESIGN.md` options (b) and (c) remain sketches. Given that re-run-from-scratch charges fuel for every replayed prefix (§8.2), (b) is the natural next step if round counts grow.
- A pure-SKI unquoter for the five-constructor encoding (the `UQ` of {`S, K, I, App, Scry`}) is not built; the driver's decoder is host-side.
- Whether to normalize paths under fuel before lookup (§8.3) is a design choice with a named cost; it is recorded, not made.

12 Appendix: the emulator as an expression

The terms drawn in Figures 1 and 5, written out. Application is juxtaposition and associates to the left; parentheses are only where association is to the right; line breaks fall at spaces and carry no meaning. Both are exactly what `expand()` produces from the named definitions under `aviary_kernel.abstraction`'s bracket abstraction, so their atom counts are algorithm-relative in the sense of §4; the atom counts below were re-checked against the printed strings. Both begin with the same fourteen atoms, `S (K (S I I)) (S (S (K S) K) (K (S I I)))`, which is `Y`: each is a fixpoint of its generator, `UQGen` and `wfGen` respectively.

`UQ`, the unquoter, 43 atoms:

```
S (K (S I I)) (S (S (K S) K) (K (S I I))) (S (K (S (S (S (S I (K S)) (K K))
(K I)))) (S (K K) (S (S (K S) (S (K (S (K S))) (S (K K)))) K)))
```

`whnfF`, the emulator, 618 atoms:

```
S (K (S I I)) (S (S (K S) K) (K (S I I))) (S (K (S (S (K S) (S (K K) (S I (K
K)))))) (S (K K) (S (K (S (S (K S) (S (K K) (S (S (S (S (S (S (K (S I I)) (S
(S (K S) K) (K (S I I))) (S (K (S (S (K S) (S (S (K S) (S (S (K S) (S (S (K
S) K) (K (S (K (S (K K))) (S (K (S (K K))) (S (K (S I)) K)))))) (K (S (K K)
(S (K (S (K K))) (S (K (S I)) K)))))) (K (S (K K) (S (K K) (S (K (S I))
K)))))))) (S (K K) (S (S (K S) (S (K K) (S (K S) (S (K (S (K S))) (S (K
K)))))) (K (S (K K) (S (K (S (S (K (S (K K))) (S (S (K S) (S (K K) (S (K S)
(S (K (S I)) K)))) (K K))))) K))))))) (K K)) (K (S (S I (K K)) (K (S (K (S
(S I (K K)))) (S (K K) (S (K (S (K (S (S I (K K)))))) (S (K (S (K K))) (S (K
(S (K (S (K (S (K (S (K K) (S (K (S I)) K))))))))) (S (K (S (K (S (K (S (K
(S I I)) (S (S (K S) K) (K (S I I))) (S (K (S (S (K S) (S (K (S I)) K)))) (S
(K (S (K K))) (S (S (K S) (S (K K) (S (K S) K))) (K (S (K (S (K K))) (S (K
(S (K K))) (S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S (K (S I)) K))))
(K K))))))))))))))) (S (S (K S) (S (K K) (S (K S) (S (K (S (K (S (K (S (K
```

```
K))) (S (K (S (K K))) (S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S (K
(S I)) K)))) (K K))))))) (S (K (S (K K))) (S (K (S (K K))) (S (K (S (K K)))
(S (S (K S) (S (K K) (S (K S) (S (K (S I)) K)))) (K K))))))))) (K (S (K (S
(K K))) (S (K (S (K K))) (S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S
(K (S I)) K)))) (K K)))))))))))))))) (K (S (S I (K K)) (K (S (K (S (S I (K
K)))) (S (K K) (S (K K) (S (K (S (K (S (K K) (S (K (S I)) K))))) (S (K (S I
I)) (S (S (K S) K) (K (S I I))) (S (K (S (S (K S) (S (K (S I)) K)))) (S (K
(S (K K))) (S (S (K S) (S (K K) (S (K S) K))) (K (S (K (S (K K))) (S (K (S
(K K))) (S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S (K (S I)) K)))) (K
K))))))))))))))))) (K (S (S I (K K)) (K (S (K (S (K (S (K K) (S (K (S I))
K))))) (S (K (S I I)) (S (S (K S) K) (K (S I I))) (S (K (S (S (K S) (S (K (S
I)) K)))) (S (K (S (K K))) (S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S
(K (S I)) K)))) (K K))))))))))))))) (S (K K) (S (K (S I)) K))))))) K)))
```

The strings are also in `figures/uq-43.txt` and `figures/whnff-618.txt`.

References

Harold Abelson, Gerald Jay Sussman, and Julie Sussman. *Structure and Interpretation of Computer Programs*. MIT Press, Cambridge, MA, 2nd edition, 1996.

Nada Amin and Tiark Rompf. Collapsing towers of interpreters. *Proceedings of the ACM on Programming Languages*, 2(POPL):33:1–33:33, January 2018. doi: 10.1145/3158140.

Kenichi Asai, Satoshi Matsuoka, and Akinori Yonezawa. Duplication and partial evaluation: For a better understanding of reflective languages. *Lisp and Symbolic Computation*, 9(2–3): 203–241, 1996.

Henk P. Barendregt. Self-interpretation in lambda calculus. *Journal of Functional Programming*, 1(2):229–233, 1991.

Alessandro Berarducci and Corrado Böhm. A self-interpreter of lambda calculus having a normal form. In *Computer Science Logic (CSL '92)*, volume 702 of *Lecture Notes in Computer Science*, pages 85–99. Springer, 1993.

Matt Brown and Jens Palsberg. Breaking through the normalization barrier: A self-interpreter for F-omega. In *Proceedings of the 43rd ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2016)*, pages 5–17, 2016.

N.Ẽ. Davis. Metacircular virtualization and practical nock interpretation. *Urbit Systems Technical Journal*, 2(1), 2025. URL https://ustj.urbit.org/article/v02-i01/metacircular-virtualization-and-practical-nock-interpretation.

Yoshihiko Futamura. Partial evaluation of computation process: An approach to a compiler-compiler. *Systems, Computers, Controls*, 2(5):45–50, 1971. Reprinted in Higher-Order and Symbolic Computation 12(4):381–391, 1999.

Jr. Guy L. Steele and Gerald Jay Sussman. The art of the interpreter, or the modularity complex (parts zero, one, and two). Technical Report AI Memo 453, MIT Artificial Intelligence Laboratory, 1978.

Neil D. Jones. What not to do when writing an interpreter for specialisation. In *Partial Evaluation: International Seminar, Dagstuhl Castle, Germany, February 1996*, volume 1110 of *Lecture Notes in Computer Science*, pages 216–234. Springer, 1996.

Neil D. Jones, Carsten K. Gomard, and Peter Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall International, 1993.

John McCarthy. Recursive functions of symbolic expressions and their computation by machine, part I. *Communications of the ACM*, 3(4):184–195, 1960.

John McCarthy, Paul W. Abrahams, Daniel J. Edwards, Timothy P. Hart, and Michael I. Levin. *LISP 1.5 Programmer's Manual*. MIT Press, Cambridge, MA, 1962.

John C. Reynolds. Definitional interpreters for higher-order programming languages. In *Proceedings of the ACM Annual Conference*, pages 717–740, 1972. Reprinted in Higher-Order and Symbolic Computation 11(4):363–397, 1998.

Brian Cantwell Smith. Reflection and semantics in Lisp. In *Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '84)*, pages 23–35, 1984.

Mitchell Wand and Daniel P. Friedman. The mystery of the tower revealed: A nonreflective description of the reflective tower. *Lisp and Symbolic Computation*, 1(1):11–38, 1988.